Analisi di Matrix-Chain Multiplication, LCS

Materia	Algoritmi
Anno	Secondo Anno
Data	@November 28, 2024
Studiato	Done

Input del problema e sottostruttura ottima

L'input del problema è una sequenza $< A_1, A_2, ..., A_n >$ di matrici di dimensioni tali che il numero di colonne della matrice A_i dev'essere uguale al numero di righe della matrice A_{i+1}. Questo garantisce che le matrici sono tra di loro moltiplicabili.

L'obiettivo del problema è **calcolare $A_1 ... A_n$ minimizzando il numero di operazioni scalari**.
Una soluzione è una parentesizzazione valida della sequenza di matrici. Alcune sequenze sono preferibili rispetto ad altre in base al costo di tale sequenza. Tale costo corrisponde al
numero di operazioni scalari, con funzione $costo(P) = \#\text{operazioni scalari}$.

Proposizione

> *Matrix-chain multiplication gode della proprietà di sottostruttura ottima.*

Partiamo da una soluzione ottima. Sia P_{opt} una soluzione ottima, ossia tale che:

$$costo(P_{opt}) \leq costo(P) \ \ \forall \text{parentesizzazione } P$$

Tale parentesizzazione è del tipo:

$$P_{opt}(A_1 ... A_n) = P_1(A_1 ... A_{i-1}) P_2(A_i ... A_n)$$

per cui $\exists P_1, P_2, \exists i \in \{ 2, ..., n \}$.

Si osserva che:

$$costo(P_{opt}) = costo(P_1) + costo(P_2) + h$$

dove h indica il numero di moltiplicazioni scalari per moltiplicare la matrice risultate da $A_1 \cdot ... \cdot A_{i-1}$ con la matrice risultate da $A_i \cdot ... \cdot A_n$.

Notiamo che h **non dipende dalle parentesizzazioni** P_1, P_2**, ma solo dal taglio** i. Questo perché il risultato della matrice non varia se fissiamo i.

Restringiamo P_{opt} ai suoi sottoproblemi $< A_1...A_{i-1} >$ e $< A_i...A_n >$, ossia P_1, P_2. Per dimostrare che matrix-chain multiplication gode della proprietà di sottostruttura ottima, è necessario dimostrare che:

- P_1 è una parentesizzazione ottima di $< A_1...A_{i-1} >$;
- P_2 è una parentesizzazione ottima di $< A_i...A_n >$.

Supponiamo per assurdo che P_1 non sia una parentesizzazione ottima. Il processo sarà analogo per P_2. Vuol dire che esiste P_1' parentesizzazione di $< A_1...A_{i-1} >$ che ha costo inferiore:

$$costo(P_1') < costo(P_1)$$

Definiamo allora la parentesizzazione P^* tale che:

$$P^*(A_1...A_n) = P_1'(A_1...A_{i-1})P_2(A_i...A_n)$$

il cui costo è:

$$costo(P^*) = costo(P_1') + costo(P_2) + h'$$

Ma per quanto visto precedentemente, $h' = h$, pertanto:

$$costo(P^*) = costo(P_1') + costo(P_2) + h < costo(P_1) + costo(P_2) + h$$

e questo è un assurdo, in quanto per ipotesi P è una soluzione ottima, pertanto il suo costo dev'essere il più basso tra tutte le parentesizzazioni P'. Tale assurdo

deriva dall'assumere che P_1 non sia una parentesizzazione ottima. Analogamente si dimostra che P_2 è una parentesizzazione ottima. Pertanto, il problema gode della proprietà di sottostruttura ottima.

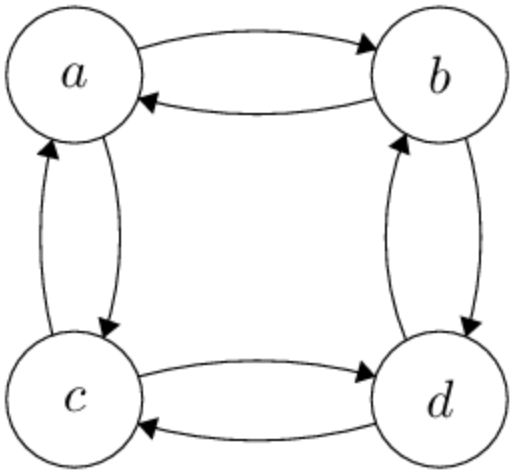
Quando è possibile applicare la programmazione dinamica

Consideriamo i seguenti problemi aventi lo stesso input: un grafo pesato $G = (V, E)$ e due vertici $u, v \in V$. Tali problemi sono:

- **unweighted shortest path**: trovare un cammino da u a v con lunghezza minima. La lunghezza di un cammino è dato dal numero di archi contenuti in tale cammino oppure dal numero di nodi meno 1 se il grafo non è pesato;
- **longest simple path**: trovare un cammino **semplice** (ossia dove non vi sono cicli) da u a v che abbia lunghezza massima.

Si può dimostrare che unweighted shortest path gode della proprietà di sottostruttura ottima. Sia C^* un cammino minimo da u a v tali che $u \neq v$. C^* può essere suddiviso in sottocammini compatibili, ossia che preso il sottocammino c_n di C^*, se l'ultimo arco di c_n arriva ad un vertice w, il sottocammino c_{n+1} ha come primo arco un arco proveniente da w. Supponiamo quindi di dividere C^* in due sottocammini compatibili c_1, c_2 e per assurdo ipotizziamo che uno dei non sia ottimo. Ad esempio, supponiamo c_1 non sia ottimo. Allora $\exists c_1' : c_1' < c_1$. Allora esiste C' da u a v tale che C' è uguale a $c_1' + c_2 < c_1 + c_2 = C^*$, il che è un assurdo poiché C^* è per ipotesi una soluzione ottima. L'assurdo deriva dall'aver ipotizzato che c_1 è non è una soluzione ottima, pertanto il problema gode della proprietà di sottostruttura ottima.

Longest simple path, tuttavia, non gode della proprietà di sottostruttura ottima. Infatti, consideriamo un grafo di vertici a, b, c, d disposti a quadrato in cui su ogni lato vi è un arco da un lato e dall'altro:



Il percorso semplice da a a d è $a \to b \to d$. Ma il cammino $a \to b$ non è il più lungo, infatti un cammino semplice più lungo è $a \to c \to d \to b$.

Questo è dovuto al fatto che non possiamo combinare le soluzioni. Questo è un problema NP-Hard.

Longest Common Subsequence (LCS)

Siano date due sequenze $X =< x_1, ..., x_m >$ e $Y =< y_1, ..., y_n >$. Trovare la sottosequenza Z tale che Z è una sottosequenza comune a X e a Y e Z ha lunghezza massima.

Ad esempio, $X =< ABCBDAB >$ e $Y =< BDCABA >$ hanno sottosequenza di lunghezza massima $Z =< BCBA >$.

Diremo che data una sequenza $X =< x_1, x_2, ..., x_n >$ e una sequenza $Z =< z_1, ..., z_n >$, diremo che Z è una sottosequenza di X se esiste una sequenza crescente di indici di $\{ i_1, ..., i_k \}$ di X tali che $x_{i_j} = z_j$. Nell'esempio precedente, abbiamo che $BCBA$ è sottosequenza di X in quanto la sequenza di indici $\{ 2, 3, 4, 6 \}$ è una sequenza crescente di indici di X tali che $X_{i_1} = X_2 = B$ e così via.

Possiamo utilizzare un approccio brute-force ed enumerare tutte le sottosequenze di X e controllare per ognuna di esse se è una sottosequenza di Y, tenendo traccia della sottosequenza comune più lunga. Ogni sottosequenza di X

corrisponde ad un sottoinsieme di indici di $\{ 1, 2, ..., m \}$. Ci sono quindi 2^m sottosequenze da controllare.

Possiamo dimostrare che il problema gode della proprietà di sottostruttura ottima.

Dimostrazione della proprietà di sottostruttura ottima

L'i-esimo prefisso di $X =< x_1, x_2, ..., x_m >$ è $X_i =< x_1, ..., x_i >$ per $i \in \{ 1, ..., n \}$. Allora vale la proposizione:

> *Siano $X =< x_1, x_2, ..., x_m >$ e $Y =< y_1, y_2, ..., y_n >$ e sia $Z =< z_1, z_2, ..., z_k >$ una LCS di X e Y. Allora:*
>
> 1. $x_m = y_n \implies z_k = x_m = y_n$ e Z_{k-1} *è LCS di* X_{m-1} *e* Y_{n-1}*;*
> 2. $x_m \neq y_n$ *e* $z_k \neq x_m \implies Z$ *è LCS di* X_{m-1} *e* Y*.*
> 3. $x_m \neq y_n$ e $z_k \neq y_n \implies Z$ *è LCS di* X *e* Y_{n-1}*.*

Nel caso 1 se $z_k \neq x_m$ allora potremmo aggiungere alla fine $x_m = y_n$ a Z e ottenere una sottosequenza comune di lunghezza $k + 1$, ma ciò è un assurdo in quanto Z è la più lunga. Z_{k-1} è comune a X_{m-1} e Y_{n-1} e ha lunghezza $k - 1$. Dimostriamo che questo prefisso è una LCS, supponendo per assurdo che ci sia W comune a X_{m-1}, Y_{n-1} di lunghezza maggiore di $k - 1$. Allora concatenando $x_m = y_n$ a W si ottiene una sottosequenza comune di X e Y la cui lunghezza è maggiore di k, che contraddice l'ipotesi che Z è LCS di X, Y.

Nel caso 2 se $z_k \neq x_m \implies Z$ è comune a X_{m-1} e Y. Se per assurdo esistesse W comune a X_{m-1} e Y di lunghezza maggiore a Z allora W sarebbe comune a X e Y e più lunga di Z, contraddicendo l'ipotesi.

Il caso 3 è simmetrico al punto 2 e si risolve allo stesso modo.

Tale teorema suggerisce un approccio ricorsivo, ossia una funzione ricorsiva di LCS.

Se considero $c[i, j]$, ossia la lunghezza della LCS di X_i, Y_j, può essere definita così:

$$c[i,j] = \begin{cases} 0 & i = 0, j = 0 \\ 1 + c[i-1, j-1] & i,j > 0 \wedge x_i = y_j \\ \max\{\, c[i, j-1], c[i-1, j] \,\} & i,j > 0 \wedge x_i \neq y_j \end{cases}$$